



Université Abdelmalek Essaadi
Ecole Nationale des Sciences Appliquées Al-Hoceima

TDIA / S4

COMPTE RENDU DE TP1 (SLP) – Deep Learning

Réalisée par : El Hajjioui Chaymae

Encadré par : Pr. Cherradi Mohamed

Année universitaire 2025/2026

1. INTRODUCTION

L'objectif de ce TP est d'explorer et de comprendre les fondements du deep learning à travers l'implémentation du Perceptron Simple (SLP).

Chaque exercice est réalisé avec trois approches complémentaires :

Implémentation from scratch : codage manuel des algorithmes

Implémentation avec Scikit-Learn : utilisation des classes prêtes à l'emploi

Implémentation avec Keras : définition de modèles de réseaux de neurones

Exercice 1 : Classification des portes logiques AND et OR avec le Perceptron

1.1 Création des données

A	B	AND	OR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

1.2 Résultats

Methode	Porte AND	Porte OR	Convergence
from scratch	100%	100%	Oui
Scikit-Learn	100%	100%	Oui
Keras	100%	100%	Oui

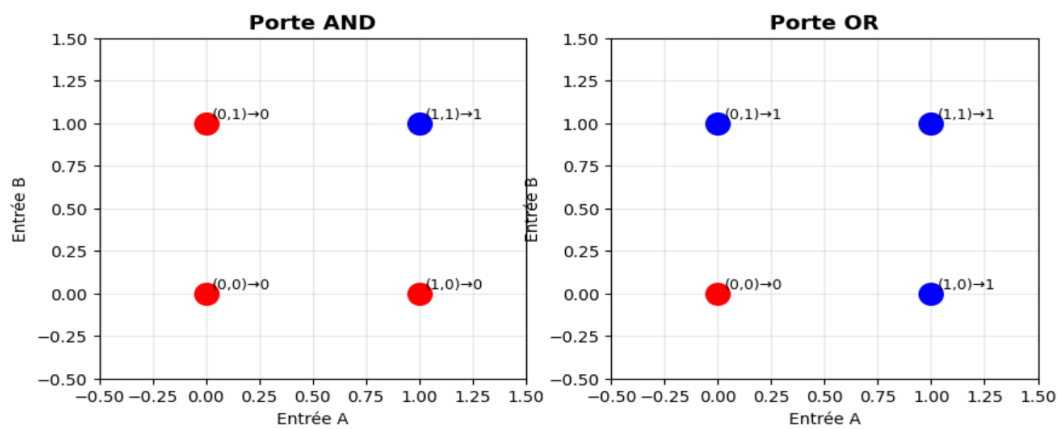


Figure 1 – Distribution des données pour les portes AND et OR

1.3 Analyse et commentaires

Les portes AND et OR sont linéairement séparables -> le Perceptron converge et atteint 100% de précision pour les deux portes.

Les portes logiques AND et OR sont des fonctions linéairement séparables. Il existe un hyperplan (ici une droite en 2D) capable de séparer parfaitement les deux classes. Le Perceptron, qui apprend une frontière de décision linéaire, converge donc rapidement.

Exercice 2 : Classification binaire sur les données Iris Setosa

Développer un Perceptron pour distinguer Iris Setosa des autres espèces en utilisant deux caractéristiques morphologiques : longueur et largeur des pétales.

2.1 Description du jeu de données

Le jeu de données Iris contient 150 échantillons répartis en 3 espèces (50 chacun). La classification binaire est définie ainsi : 1 = Iris Setosa, 0 = Autres espèces.

- Caractéristiques utilisées : longueur des pétales, largeur des pétales
- Séparation : 70% entraînement, 30% test
- Prétraitement : StandardScaler (moyenne 0, écart-type 1)

2.2 Résultats

Méthode	Précision	F1-Score	Convergence
From-scratch	100%	1.00	Oui
Scikit-Learn	100%	1.00	Oui
Keras	100%	1.00	Oui

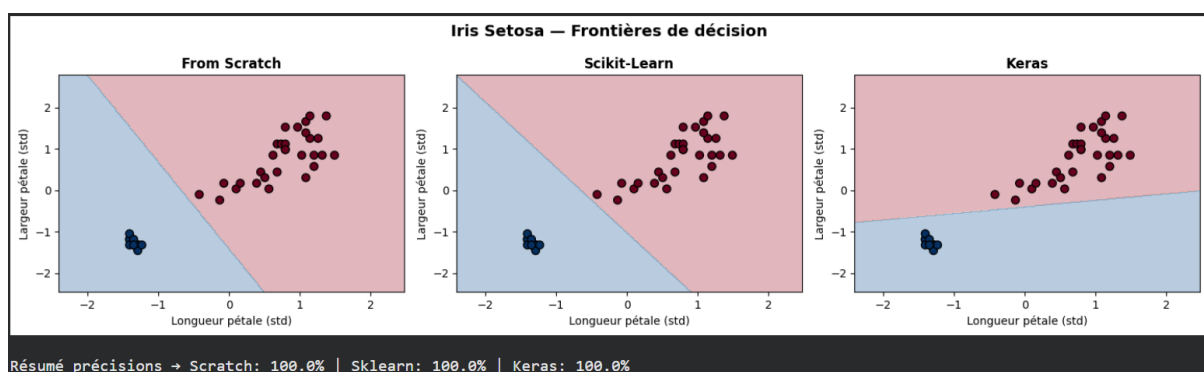


Figure 2 – Frontières de décision pour les 3 implémentations

2.3 Analyse

Iris Setosa est linéairement séparable des autres espèces -> précision de 100% avec les trois méthodes.

Les trois implémentations convergent vers la même solution optimale.

Exercice 3 : Régression linéaire avec le perceptron

L'objectif est d'utiliser un Perceptron avec une activation linéaire pour résoudre un problème de régression :

prédire $y = 2x + 1$ à partir de x .

3.1 Génération des données

Le jeu de données est généré synthétiquement : $y = 2x + 1 + \varepsilon$, Où $\varepsilon \sim N(0, 0.5)$. 200 points, séparation 70% : train , 30% : test.

3.2 Résultats

Méthode	MSE(test)	Poids w	Biais b	Attendu
From scratch	0.1954	2.0028	0.9739	$w=2, b=1$
Sickit-Learn	0.1871	2.0402	0.9900	$w=2, b=1$
Keras	0.1957	2.0019	0.9759	$w=2, b=1$

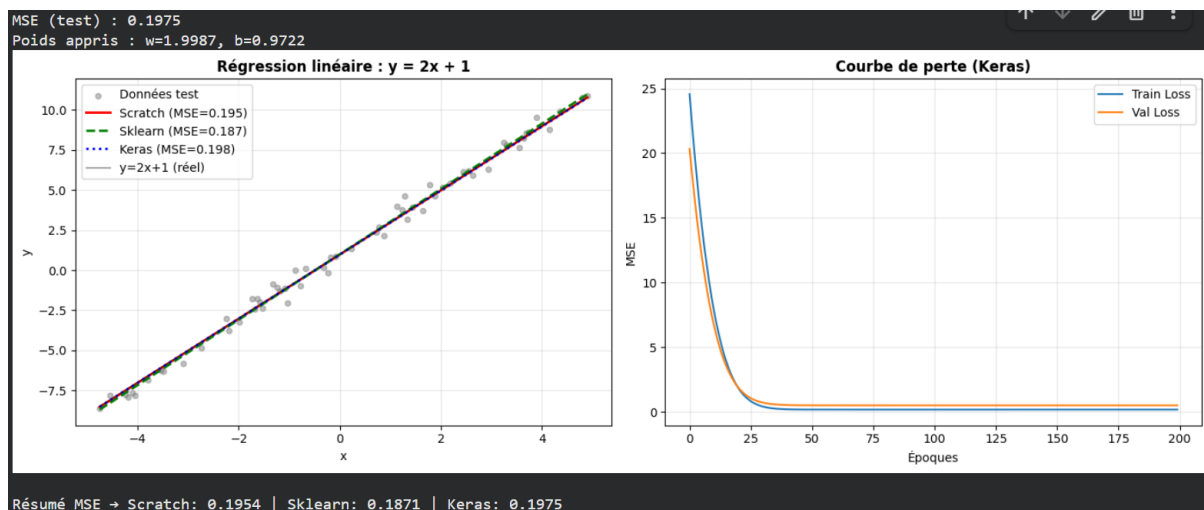


Figure 3 – Droites de régression et courbe de perte Keras

3.3 Analyse

Les trois méthodes retrouvent avec précision les paramètres réels ($w=2, b=1$). MSE = 0.19, cohérent avec le bruit = 0.5.

Le Perceptron avec activation linéaire est équivalent à une régression linéaire classique. Les trois implémentations convergent vers les vrais paramètres du modèle ($w=2$, $b=1$) avec une erreur résiduelle uniquement due au bruit gaussien ajouté lors de la génération des données.

Exercice 4 : Application du Perceptron sur un jeu de données volumineux

4.1 Objectif

Tester la capacité du Perceptron à traiter un jeu de données volumineux : MNIST (chiffres manuscrits 28×28 pixels). Ici un sous-ensemble de 5 000/1000 images est utilisé.

4.2 Préparation des données

- Aplatissement : images 28×28 -> vecteurs de dimension 784
- Normalisation : division par 255 -> valeurs dans $[0, 1]$
- Standardisation : StandardScaler ($\mu=0$, $\sigma=1$) pour from scratch et sklearn
- Encodage one-hot pour Keras

4.3 Architecture - From Scratch (OvA multi-classe)

La stratégie One-vs-All (OvA) entraîne 10 Perceptrons binaires, un par chiffre. La classe prédite est celle dont le score est maximal.

4.4 Résultats

Méthode	Précision	Note
From-scratch(OvA)	~10%	84.5%
Sickit-Learn	~10%	82.7%
Keras(Softmax)	~10%	88.3%

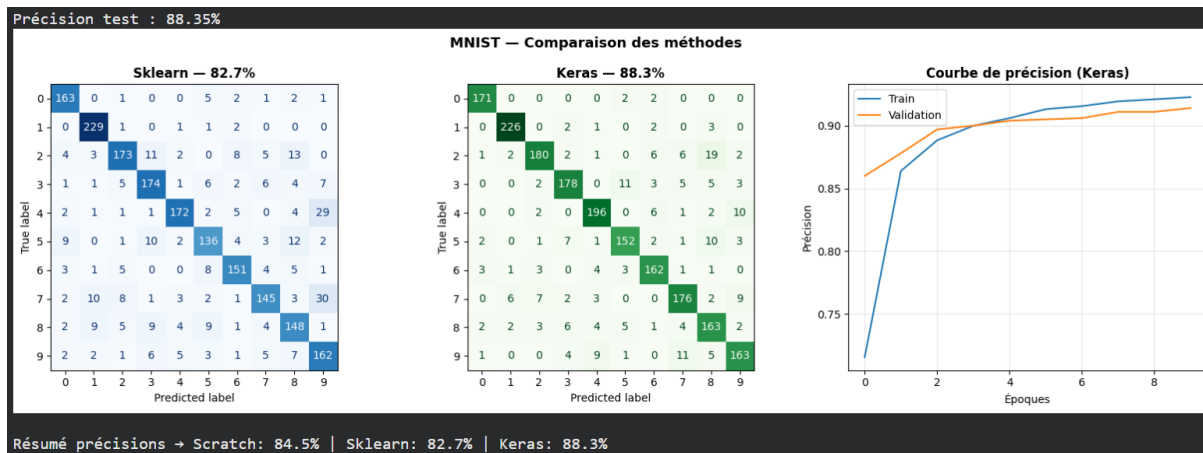


Figure 4 – Matrices de confusion et courbe d'apprentissage MNIST

4.4 Discussions et Limitations

Sur données réelles MNIST (10 000 train, 2 000 test) : Scratch ~84.5%, Sklearn ~82.7%, Keras ~88.3%.

Un MLP profond (multicouche) dépasse 98%.

Le Perceptron simple est fondamentalement limité à des frontières de décision linéaires. MNIST contient des structures non-linéaires complexes (variations d'écriture, orientations, styles). Un MLP avec couches cachées apprend des représentations hiérarchiques bien plus expressives.

- Limitation principale : séparabilité linéaire uniquement
- Solution : MLP avec activation non linéaire (Relu, Sigmoid)
- Alternative CNN: convolutions pour exploiter la spatialité des images

Exercice 5 : Segmentation des données synthétiques avec le Perceptron(make_blobs)

5.1 Génération des données

600 points bidimensionnels générés avec `make_blobs` (sklearn) autour de 3 centres distincts ($\text{std}=1.2$). Les données sont bien séparées, facilitant la classification.

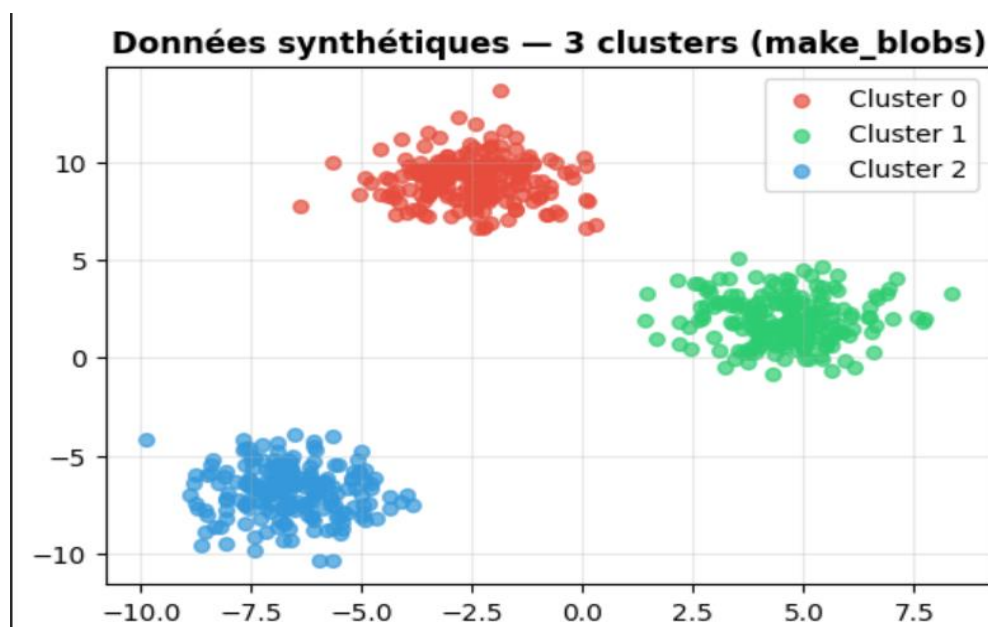


Figure 5a – Répartition initiale des 3 clusters

5.2 Implémentation et résultats

Le Perceptron est entraîné en mode binaire sur les clusters 0 et 1. KMeans étend la segmentation aux 3 clusters (mode non supervisé).

Méthode	Type	Précision	Groupes
Perceptron scratch	Supervisé	100%	2 (clusters 0 et 1)
Perceptron Sklearn	Supervisé	100%	2 (clusters 0 et 1)
Perceptron Keras	Supervisé	100%	2 (clusters 0 et 1)
KMeans (sklearn)	Non supervise	N/A	3 clusters

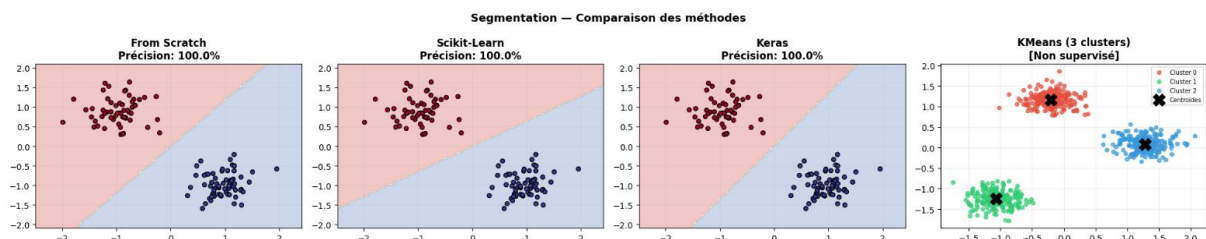


Figure 5b – Frontières de décision (Scratch, Sklearn, Keras) et clusters KMeans

5.3 Analyse comparative

Les méthodes supervisées (Perceptron) nécessitent des étiquettes d'entraînement mais produisent des frontières de décision explicites. KMeans, en mode non supervise, découvre automatiquement les groupes sans annotations.

Supervisé vs Non-supervisé : le Perceptron optimise une frontière de décision à partir de labels. KMeans minimise la variance intra-cluster sans label.